Guia Prático Guiado: Persistência Alternativa NoSQL (MongoDB)

Academia Java Backend — Aula 03 (Prática Guiada Extra)

1. Visão Geral e Objetivos Pedagógicos

Nesta atividade guiada complementar, vocês explorarão uma alternativa NoSQL orientada a documentos utilizando o **Spring Data MongoDB**. O objetivo é vivenciar a flexibilidade do armazenamento NoSQL e comprovar a robustez da **Clean Architecture**, observando como as camadas superiores da aplicação (Controller e DTOs) permanecem praticamente inalteradas.

> 💡 **DECISÃO DE EXECUÇÃO: ZERO SETUP & ZERO FRICÇÃO**
> Para viabilizar esta prática sem instalar o MongoDB localmente e sem exigir Docker neste estágio (que será abordado na Aula 07), utilizamos a biblioteca **mongo-java-server (MemoryBackend)**. Ela simula um servidor MongoDB completo em memória dentro da própria JVM, garantindo 100% de compatibilidade com o Spring Data MongoDB.

2. Passo 1: Sincronização de Tags e Isolamento da Branch Git

Para garantir que a prática parta do estado limpo pré-persistência e não contamine a branch principal (**main**), siga rigorosamente a sequência de comandos abaixo no terminal:

```
# 1. Sincronizar e garantir todas as tags do repositório remoto
git fetch --tags

# 2. Efetuar checkout OBRIGATÓRIO no checkpoint da Aula 02 (API Design & DTOs)
git checkout checkpoint/aula-02-api-design

# 3. Criar e mudar para a branch isolada do exercício NoSQL
git checkout -b feature/mongo-persistence
```

📌 **Nota Importante:** Ao realizar o checkout da tag **checkpoint/aula-02-api-design**, garantimos que o projeto está exatamente no estado de API REST com DTOs e lista em memória, sem as anotações do JPA/H2 que adicionamos na Aula 03 oficial.

3. Passo 2: Adicionar Dependências no pom.xml

Adicione as seguintes dependências no arquivo **pom.xml** do projeto:

```
<!-- Spring Data MongoDB Starter -->
<dependency>
```

```
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-mongodb</artifactId>
</dependency>

<!-- Servidor MongoDB Simulado em Memória (Zero Setup) -->
<dependency>
    <groupId>de.bwaldvogel</groupId>
    <artifactId>mongo-java-server</artifactId>
    <version>1.47.0</version>
</dependency>
```

4. Passo 3: Mapeamento da Coleção (MovieDocument.java)

No MongoDB, não utilizamos tabelas nem `@Entity`. Mapeamos documentos em coleções com a anotação **@Document** e usamos **String id** para o identificador `ObjectId` de 24 carateres:

```
package pt.rumos.mymovies.model;

import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.mapping.Document;
import org.springframework.data.mongodb.core.mapping.Field;

@Document(collection = "movies")
public class MovieDocument {

    @Id
    private String id; // ObjectId do MongoDB (24 caracteres hex)
    private String title;
    private String director;

    @Field("release_year")
    private Integer year;

    private Double rating;

    public MovieDocument() {}

    public MovieDocument(String title, String director, Integer year, Double rating) {
        this.title = title;
        this.director = director;
        this.year = year;
        this.rating = rating;
    }

    // Getters e Setters...
}
```

5. Passo 4: Configurar o Mongo Memory Server (MongoMemoryConfig.java)

Crie a classe de configuração para subir o servidor MongoDB em memória na inicialização do Spring Boot:

```
package pt.rumos.mymovies.config;

import de.bwaldvogel.mongo.MongoServer;
import de.bwaldvogel.mongo.backend.memory.MemoryBackend;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.data.mongodb.core.MongoTemplate;
import org.springframework.data.mongodb.core.SimpleMongoClientDatabaseFactory;
import com.mongodb.client.MongoClients;
import java.net.InetSocketAddress;

@Configuration
public class MongoMemoryConfig {

    @Bean
    public MongoTemplate mongoTemplate() {
        MongoServer server = new MongoServer(new MemoryBackend());
        InetSocketAddress serverAddress = server.bind();
        String connectionString = "mongodb://" + serverAddress.getHostName() + ":" +
serverAddress.getPort() + "/mymovies";
        return new MongoTemplate(new
SimpleMongoClientDatabaseFactory(MongoClients.create(connectionString), "mymovies"));
    }
}
```

6. Passo 5: Criar a Interface MongoRepository

Crie o repositório estendendo **MongoRepository<MovieDocument, String>**:

```
package pt.rumos.mymovies.repository;

import org.springframework.data.mongodb.repository.MongoRepository;
import pt.rumos.mymovies.model.MovieDocument;
import java.util.List;

public interface MovieDocumentRepository extends MongoRepository<MovieDocument, String> {
    List<MovieDocument> findByTitleContainingIgnoreCase(String title);
}
```

7. Passo 6: Adaptação da Camada Controller

No **MovieService**, injetamos o `MovieDocumentRepository` e adaptamos a conversão para DTOs. No **MovieController**, a única alteração necessária é ajustar as assinaturas dos parâmetros de rota de **@PathVariable Long id** para **@PathVariable String id**.

> **INSIGHT DE ARQUITETURA (CLEAN ARCHITECTURE & DTOs)**
> Note o reduzidíssimo esforço na camada Web (`MovieController` e `MovieRequest`) ao alterar todo o paradigma de persistência (de SQL para NoSQL Documental). Como os DTOs desacoplam o contrato da API do modelo de armazenamento, os clientes HTTP continuam a interagir com a mesma estrutura JSON. A única mudança externa perceptível é o formato do ID (de inteiro sequencial para hash hexadecimal).

```
// MovieController.java (Muda apenas a tipagem do ID para String). Altere outros pontos que
// utilizem o id

@GetMapping("/{id}")
public ResponseEntity<MovieResponse> findById(@PathVariable String id) {
    return ResponseEntity.ok(service.findById(id));
}

@DeleteMapping("/{id}")
public ResponseEntity<Void> delete(@PathVariable String id) {
    service.delete(id);
    return ResponseEntity.noContent().build();
}
```

8. Passo 7: Adaptação da Camada Service

Com a transição para o **MongoDB**, o tipo de dados do identificador único altera-se de **Long** (sequencial numérico do SQL) para **String** (representando o ObjectId de 24 carateres hexadecimais gerado nativamente pelo MongoDB).

A camada de serviço passa a interagir com a interface MovieDocumentRepository (que estende `MongoRepository

```
@Service public class MovieService {
   private final MovieDocumentRepository repository;

   // Injeção de dependência via Construtor (Boa Prática)
   public MovieService(MovieDocumentRepository repository) {
     this.repository = repository;
   }

   // Refatorar apenas os pontos onde o id Long aparece para id String.
   …
```

9. Passo 8: Teste no Bruno e a Volatilidade em Memória

1. Execute a aplicação e faça um **POST /api/movies** no Bruno.
2. Observe a resposta HTTP 201 Created com o ID no formato String: **"id": "65f8a3b11c9d2e001a2b3c4d"**.
3. **Teste do Restart:** Reinicie a aplicação no IntelliJ e faça um GET /api/movies. A lista retornará vazia, demonstrando que a base de dados simulada em memória reseta a cada reinicialização da JVM.

10. Passo 9: Salvaguarda da Branch e Retorno Seguro à Main

Ao concluir o exercício, salve o trabalho na branch da funcionalidade e retorne para a branch **main** para manter o percurso oficial do curso baseado em SQL (Spring Data JPA + H2):

```
# 1. Adicionar e comitar as alterações na branch NoSQL
git add .
git commit -m "feat: adiciona solucao de persistencia alternativa com mongodb"

# 2. Retornar à branch main oficial (SQL / JPA / H2)
git checkout main
```